

CS 300 - Advanced Programming
Assignment 3: Web application programming

Spring 2026

Lahore University of Management Sciences
School of Science & Engineering

April 15, 2026

Contents

1	Project Overview	4
1.1	Key Objectives	4
1.2	Tech Stack	4
2	Folder Structure	4
3	Setup & Installation	5
3.1	Prerequisites	5
3.2	Install Dependencies	5
3.3	Required Packages	6
3.4	Environment Variables	6
3.5	Start the Application	7
4	Application Routes & Pages	7
4.1	Route Summary	7
4.2	Page Navigation Flow	7
4.3	Page Descriptions	8
5	Game Rules & Mechanics	9
5.1	Board Layout	9
5.2	Turn Order & Dice	10
5.3	Movement Rules	10
5.4	Captures	10
5.5	Winning & Coin Rewards	10
5.6	Special Cases	11
6	Architecture & Request Flow	11
6.1	Navbar (Persistent After Login)	12
6.2	Component Breakdown	12
7	Socket.IO Event Reference	12
8	Database Models (MongoDB)	12
8.1	User Schema	12
8.2	Game Schema	12
9	REST API (Backend)	13
10	Gameplay Page Detail (/newgame/:game_id)	13
10.1	Top Bar	13
10.2	Left Sidebar	13
10.3	Board Area	13
10.4	Right Sidebar	14
10.5	Victory Overlay	14
10.6	Common Pitfalls	14
11	Requirements Checklist	14
11.1	Frontend (React)	15
11.2	Backend (Node.js/Express)	15
11.3	Database (MongoDB)	15
12	Submission Instructions	15

12.1 Step by Step Submission guide	15
12.2 Submissions Running Process	16
13 Tentative Grading Criteria	16
13.1 Code Quality & Architecture (20 points)	16
13.2 Frontend Implementation (20 points)	16
13.3 Backend Implementation (20 points)	16
13.4 Game Logic & Real-Time Features (25 points)	17
13.5 Database & Persistence (10 points)	17
13.6 Requirements Checklist Compliance (5 points)	17
13.7 Deductions	17

1 Project Overview

This project is a real-time, browser-based multiplayer Ludo game built as a full-stack web application. Up to four players can join a shared lobby, play on a classic 15×15 Ludo board, and communicate through an in-game live chat. All game state is synchronised between clients using Socket.IO. **The game must also persist through disconnects or page reloads.**

The project is divided into two main parts:

- **client/**: React frontend (Vite). Handles all UI, routing, and socket event listening.
- **server/**: Node.js/Express backend. Manages game logic, Socket.IO events, and database operations.

The **design/** folder contains starter HTML/CSS files that serve as visual references. You must convert these into React components, they are not served directly.

1.1 Key Objectives

- Real-time multiplayer gameplay for 2–4 players using Socket.IO
- Persistent user accounts with login, signup, and profile management
- Leaderboard tracking player coin balances and match history
- Live in-game chat with system notifications (captures, finishes, etc.)
- Full game state synchronization with no polling
- AI-controlled tokens for disconnected players (random moves; no intelligent strategy required)

1.2 Tech Stack

Layer	Technology	Role
Frontend	React (Vite)	UI, routing, board rendering, socket listener
Real-time	Socket.IO	Bidirectional events: moves, chat, turn sync
Backend	Express.js	REST API, room management, auth
Database	MongoDB + Mongoose	Users, Game history

2 Folder Structure

The repository ships with the following layout. You must populate the empty stub folders as described below.

```

/
|-- .gitignore
|-- client/
|-- design/                                # Static HTML/CSS reference -- do NOT serve
|   |-- index.html                        # Landing page (no auth required)
|   |-- index.css
|   |-- login.html                        # Login form
|   |-- login.css
|   |-- signup.html                      # Registration form

```

```
| |-- signup.css
| |-- home.html           # Main dashboard (requires auth)
| |-- home.css
| |-- lobby.html          # Game lobby waiting area
| |-- lobby.css
| |-- game.html           # Active game interface
| |-- game.css
| |-- leaderboard.html    # Global rankings
| |-- leaderboard.css
| |-- history.html        # User game history
| |-- history.css
| |-- update-profile.html # Profile settings
| |-- update-profile.css
| |-- styles.css          # Global styles (copy to client/src/index.css)
|-- server/
| |-- app.ts              # Express app: middleware, routes
| |-- server.ts           # HTTP + Socket.IO bootstrap
| |-- config.env          # Environment variables (do not commit)
| |-- package.json
| |-- controllers/        # gameController.js, authController.js ...
| |-- models/             # User.js, Game.js (Mongoose schemas)
| |-- routes/             # /api/auth, /api/rooms, /api/stats ...
| |-- utils/              # gameLogic.js, diceRoller.js, aiPlayer.js
```

3 Setup & Installation

3.1 Prerequisites

Requirement	How to verify
Node.js ≥ 18	<code>node -v</code>
npm ≥ 9	<code>npm -v</code>
MongoDB (local or Atlas)	<code>mongosh</code> or Atlas dashboard

3.2 Install Dependencies

Open two separate terminal windows and run the following commands:

```
# Terminal 1 -- Server
cd server
npm install

# Terminal 2 -- Client
cd client
npm create vite@latest .
```

```

PS E:\CS300 2026 Spring\PAs\pa3-mern\client> npm create vite@latest .

> npx
> create-vite .

|
|   Current directory is not empty. Please choose how to proceed:
|   Remove existing files and continue
|
|   Select a framework:
|   React
|
|   Select a variant:
|   TypeScript
|
|   Install with npm and start now?
|   Yes
|
|   Scaffolding project in E:\CS300 2026 Spring\PAs\pa3-mern\client...
|
|   Installing dependencies with npm...
|
added 172 packages, and audited 173 packages in 25s
49 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
|   Starting dev server...
|
> client@0.0.0 dev
> vite

VITE v8.0.8 ready in 310 ms
→ Local: http://localhost:5173/

```

Figure 1: Frontend setup screenshot

3.3 Required Packages

Folder	Package	Purpose
server	express	HTTP server & REST routes
server	socket.io	Real-time bidirectional events
server	mongoose	MongoDB ODM
server	cors	Allow requests from React dev server
server	dotenv	Load environment variables from <code>config.env</code>
server	nodemon (dev)	Auto-restart server on file changes
client	socket.io-client	Connect to Socket.IO server from React
client	react-router-dom	Client-side routing between pages
client	axios (optional)	HTTP requests for REST endpoints

3.4 Environment Variables

Environment variables are used to store configuration values outside the source code. This helps keep sensitive information such as database credentials secure and makes the application easier to configure for different environments (e.g., development and production).

Create a file named `config.env` inside the `server/` directory:

```
PORT=8000
MONGO_URI=mongodb+srv://<user>:<pass>@cluster0.xxxxx.mongodb.net/ludo
NODE_ENV=development
```

Load it at the very top of `server.js`:

```
require('dotenv').config({ path: './config.env' });
```

This ensures that all environment variables are available throughout the application via `process.env`.

3.5 Start the Application

```
# Terminal 1 -- start backend on port 8000
cd server
npm run dev

# Terminal 2 -- start frontend on port 5173
cd client
npm run dev
```

Navigate to <http://localhost:5173> in your browser.

4 Application Routes & Pages

Each URL maps to a React page component. The `design/` HTML files are your visual reference for each page.

4.1 Route Summary

Route	Page / Purpose
/	Landing page — two options: Login and Sign Up
/login	Login form: username + password
/signup	Registration form: username, DOB, password, confirm password
/home	Main dashboard: Play, Leaderboard, History links
/newgame/lobby	Waiting lobby: players join before game starts
/newgame/:game_id	Active gameplay page (real-time via Socket.IO)
/history	Display history for the logged-in user
/leaderboard	Global rankings by coin balance
/update-profile	Edit username, DOB, and password

4.2 Page Navigation Flow

The application follows a linear authentication flow, then branches into game and profile management:

- **Unauthenticated Flow:** / (index) → /login or /signup
- **Authenticated Entry:** After login/signup redirect to /home
- **From Dashboard:**

- Play Game → /newgame/lobby → /newgame/:game_id
- View Rankings → /leaderboard → back to /home
- View History → /history → back to /home
- Dropdown: Update Profile → /update-profile → back to /home
- Dropdown: Logout → redirect to /

- **In-Game:** Players remain on /newgame/:game_id until game ends
- **Navigation Bar:** Clicking LUDO logo from any logged-in page returns to /home

4.3 Page Descriptions

4.3.1 / — Landing Page

Welcome screen. Displays the game name and two navigation buttons: Login and Sign Up. This is the only page accessible to unauthenticated users (along with /login and /signup).

4.3.2 /login — Login

- Form fields: Username, Password
- On success: redirect to /home; store an auth token in a browser cookie to identify the session and user
- Full password encryption is not required; plain username/password matching against the database is sufficient for this project

4.3.3 /signup — Registration

- Form fields: Username (unique), Date of Birth (date picker), Password, Confirm Password
- Validate that passwords match and the username is not already taken
- Every new account starts with **100 coins**
- On success: redirect to /home

4.3.4 /home — Dashboard

Serves as the main hub after login. Navigation options: Play (goes to lobby), Leaderboard, History.

4.3.5 /newgame/lobby — Waiting Lobby

- Players wait here until the host starts the game
- The **Start Game** button is disabled until at least two players have joined
- The game can start with 2, 3, or 4 players
- Once any player clicks Start, all clients are redirected to /newgame/:game_id

4.3.6 /history — Match History

Displays all previously played games for the logged-in user. Each entry shows: Game ID, total players, finishing order (1st through 4th), coins earned.

Format: *"Game #101 — 4 players: You, Ali, Sara, Ahmed — 100 coins earned"*

Use a dash (–) for missing player slots in games with fewer than 4 players.

4.3.7 /leaderboard — Leaderboard

- Shows all players ranked by coin balance (highest first)
- Tiebreaker: if two players have equal coins, the one who earned them in fewer games ranks higher
- Each row shows: rank, username, total games played, coin balance
- Includes a search bar to filter players by username

4.3.8 /update-profile — Profile Settings

- Form fields: Username, Password, Date of Birth (date picker)
- Clicking Save updates the user record in MongoDB

Do not access the database directly from the frontend. All data operations must go through the server via API calls or socket events.

5 Game Rules & Mechanics

Ludo is a classic strategy board game for 2–4 players. This section documents all rules used in this implementation.

5.1 Board Layout

The board is a 15×15 grid divided into distinct zones:

Zone	Description
Home Zones (4)	One 6×6 coloured area per player. All tokens start here.
Main Track	52 squares forming a clockwise outer loop (13 per side).
Home Stretch	Each player has a coloured path of 5 squares leading from the main track to the centre. A total dice count of 6 is required to complete this path: 5 moves along the stretch and 1 final move to enter the centre.
Centre Finish	Final destination. A token must land here with an exact roll.
Safe Squares	Starred (★) squares on the outer track where tokens cannot be captured.
Start Squares	The specific outer-track square each colour enters on their first move.

Player colour and home zone positions:

Colour	Home Zone
Red	Top-left
Blue	Top-right
Green	Bottom-left
Yellow	Bottom-right

5.2 Turn Order & Dice

- Turns cycle clockwise: **Red** → **Blue** → **Green** → **Yellow** → **Red** ...
- Each turn the active player rolls one die (value 1–6).
- Rolling a **6** grants an extra turn immediately after the current move.
- A player cannot skip their turn; they must roll.
- If a player does not roll within **20 seconds**, the server auto-rolls and moves a random valid token on their behalf.

5.3 Movement Rules

- A token can only leave its Home Zone when the player rolls a **6**.
- Once on the main track, a token advances clockwise by the number rolled.
- If multiple tokens are movable, the player must click the token they wish to move.
- A token enters its Home Stretch after completing the full outer loop.
- A token must reach the centre finish with an **exact roll**. Over-rolling does nothing — the token stays in place.

5.4 Captures

- Landing on a square occupied by an opponent's token (on a non-safe square) captures it.
- The captured token is returned to its Home Zone and must re-enter on a future roll of 6.
- Capturing does **not** grant a bonus turn.
- Safe squares (★) and Home Stretch squares are exempt from capture.

5.5 Winning & Coin Rewards

The first player to move all four tokens to the centre finish wins. Final standings are recorded in order of completion. Coins are awarded as follows:

Game Type / Position	Coins Awarded
4-player game — 1st	100 coins
4-player game — 2nd	50 coins
4-player game — 3rd	25 coins
4-player game — 4th	0 coins
3-player game — 1st	50 coins
3-player game — 2nd	25 coins
3-player game — 3rd	0 coins
2-player game — 1st	25 coins
2-player game — 2nd	0 coins

5.6 Special Cases

5.6.1 No Valid Moves

If a player rolls a number that cannot legally move any of their tokens (e.g., all tokens are in the Home Zone and the roll is not 6, or all movable tokens would over-shoot the finish), the turn simply passes to the next player.

5.6.2 Player Disconnect

If a player disconnects during the game, an AI takes control of their tokens for the remainder of the match. The AI performs basic actions such as auto-rolling the dice and moving a random valid token each turn; no advanced strategy is required. When the player reconnects, control is returned from the AI. However, any moves made by the AI during the disconnection period cannot be changed or undone.

6 Architecture & Request Flow

All authoritative game state lives on the server. Clients send action events; the server validates them, updates state, and broadcasts the new state to every player in the room.

1. Player opens / and is shown Login / Sign Up options.
2. After authentication (You can save a token in the browser's cookies to keep the user logged in), the player is redirected to /home.
3. Clicking **Play** navigates to /newgame/lobby. The client emits `room:join`; the server assigns a colour and notifies all lobby members.
4. When the host clicks **Start Game**, the server emits `game:start` to everyone. All clients navigate to /newgame/:game_id.
5. During gameplay, every action (roll, move, chat) is a socket emit. The server validates, mutates state, and broadcasts `game:state` back to all players.
6. When a player wins, the server emits `game:over` with final standings. All clients display the Victory overlay.

6.1 Navbar (Persistent After Login)

A persistent navigation bar appears on every page once the user is logged in:

- **Left side:** game icon + name; clicking navigates to /home
- **Right side:** coin balance, username dropdown with links to Update Profile and Logout

6.2 Component Breakdown

The UI should be divided into reusable components so that each part of the system has a clear responsibility. For example, the main board can be handled by a **LudoBoard** component, while individual game pieces can be managed using a **Token** component.

Other parts of the interface, such as dice rolling and turn control, can be separated into a **DicePanel**. Player-related information can be displayed using a **PlayerCard**, and communication features like chat can be handled by a **ChatBox**.

This modular structure helps keep the code organized, maintainable, and easier to extend.

7 Socket.IO Event Reference

All real-time communication uses Socket.IO rooms identified by `game_id`. No game action should ever be performed through REST — use socket events exclusively during gameplay. Also use socket.IO for the implementation of Chat functionality in the game.

Always generate the dice roll on the server. Never accept a roll value sent by the client — this prevents cheating. Store the current roll in the room state and validate any move against it.

8 Database Models (MongoDB)

The backend uses **Mongoose** to connect the application with MongoDB. The provided schemas are only a base reference, and you are free to modify or extend them based on project requirements. All data models should be defined using Mongoose schemas to ensure structured storage, validation, and consistency across the application.

8.1 User Schema

```
// server/models/User.js
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  dob: { type: Date, required: true },
  coins: { type: Number, default: 100 },
  total_played: { type: Number, default: 0 },
});
```

8.2 Game Schema

```
// server/models/Game.js
const gameSchema = new mongoose.Schema({
  total_players: { type: Number, required: true },
  players: [{
```

```
    user_id:      mongoose.Schema.Types.ObjectId,
    username:     String,
    color:        { type: String, enum: ['red', 'blue', 'green', 'yellow'] },
    rank:         Number,    // 1 = winner
    coins_earned: Number,
  }],
  status:        { type: String, enum: ['waiting', 'playing', 'finished'],
    default: 'waiting' },
  started_at:    Date,
  finished_at:   Date,
});
```

9 REST API (Backend)

REST endpoints are used for authentication, room setup, and stats retrieval. All in-game actions must use Socket.IO events, not REST calls. You are free to design and implement the backend according to your approach however, it must follow a proper structure and best practices. The implementation should adhere to the prescribed format, including separation into controllers, models, routes, and other relevant components.

10 Gameplay Page Detail (/newgame/:game_id)

The gameplay page is the most important part of the application. Everything on this page must stay in sync across all connected clients via Socket.IO — no polling, no refreshes.

10.1 Top Bar

- Room ID and Game Mode (e.g., “Classic 4-player”)
- Turn timer: counts down from 20 seconds per turn; auto-rolls and moves randomly if it expires
- Leave Game button: triggers `room:leave` and returns the player to `/home`

10.2 Left Sidebar

Dice Panel

- Displays current die face (value 1–6)
- Roll button — enabled only on the active player’s turn
- Recent roll history (last 5 rolls)

Players Panel

- One card per player showing: name, colour dot, “Your Turn” badge (active player only)
- Stats: tokens on board — tokens in home — tokens finished
- Progress bar showing percentage of tokens completed

10.3 Board Area

- Central 15×15 Ludo board clearly divided into home zones, main track, safe squares, start squares, and home stretches
- Tokens are clickable only when they represent valid moves for the active player

- Highlight selectable tokens after a roll (e.g., glow or border animation)
- Optionally: animate token movement and show a capture flash

10.4 Right Sidebar

Live Chat

- Real-time messages from all players in the room
- System messages for game events: “Red captured Blue’s token”, “Sara finished all tokens!”
- Text input + Send button; optional quick-react emoji buttons

Game Log

- Chronological list of all game events with timestamps
- Events: dice rolls, token moves, captures, finishes

Scores & Stats

- Per-player: tokens finished, captures made, turns taken, number of 6s rolled
- Live standings showing current ranking during the game

10.5 Victory Overlay

- Full-screen overlay shown when `game:over` is received
- Displays winner name and colour, final standings, summary stats (captures, turns, duration)
- Buttons: **Play Again** (back to lobby) and **Main Menu** (back to `/home`)

10.6 Common Pitfalls

Do not store the dice roll only in client state. Generate it server-side, store in the room object, and emit the result to all clients.

Do not query MongoDB directly from React components. All database operations must go through the Express server.

Use React Context (`AuthContext`, `GameContext`) to avoid prop-drilling socket instances and user data through many component layers.

Socket.IO connections persist across React re-renders. If you call `io()` inside a component, you will create a new connection on every render. Always use a singleton module.

11 Requirements Checklist

Use this checklist to confirm your submission meets all expectations.

11.1 Frontend (React)

- ☐ React functional components throughout — no class components
- ☐ `useState` and `useEffect` used appropriately for state and side effects
- ☐ `useContext` or custom hooks used to share auth and game state; browser cookies may be used to store tokens
- ☐ React Router for all navigation (no `window.location` hacks)
- ☐ No page reloads during active gameplay
- ☐ Basic form validation (required fields, password confirmation, min length)
- ☐ Minimal, usable styling — at least use the provided CSS as a baseline

11.2 Backend (Node.js/Express)

- ☐ Socket.IO for all real-time events — no polling
- ☐ Server-side game logic (move validation, capture detection, win check)
- ☐ AI auto-play for disconnected players
- ☐ 20-second auto-roll timer enforced server-side
- ☐ Game results and coin updates persisted to MongoDB when the game ends

11.3 Database (MongoDB)

- ☐ User schema: username (unique), password, dob, coins (default 100), total_played
- ☐ Game schema: total_players, players array with rank and coins_earned, status, timestamps
- ☐ History endpoint returns correct data for the logged-in user
- ☐ Leaderboard sorted by coins descending with correct tiebreaker logic

You are free to use Tailwind CSS instead of the provided `styles.css` if you prefer — but the layout and UX of the design files must be maintained.

12 Submission Instructions

Follow these steps carefully to prepare your submission:

12.1 Step by Step Submission guide

- Remove the `node_modules` folder from both `client` and `server` directories.
- Compress the entire project folder into a `.zip` file.
- Do not use formats like `.rar` or `.7z`.
- Rename the zip file using your roll number (e.g., `28100123.zip`).
- Submit the final `.zip` file.

12.2 Submissions Running Process

1. Extract your ZIP file
2. Run `npm install` in both `client/` and `server/`
3. Configure environment variables (`config.env`)
4. Start the server and client using `npm run dev`

If your code does not run after these steps, you will lose marks. Ensure your project builds and runs cleanly from scratch with only `npm install` required.

13 Tentative Grading Criteria

Your submission is graded out of a total of **100 points**, distributed as follows:

13.1 Code Quality & Architecture (20 points)

- **(5 points)** Proper TypeScript usage with type definitions; no `any` types
- **(5 points)** Clean code structure: well-organized files, meaningful naming, comments (**where needed**)
- **(5 points)** Separation of concerns: controllers, models, routes, utilities properly separated
- **(5 points)** DRY principle: no significant code duplication; reusable components and functions

13.2 Frontend Implementation (20 points)

- **(4 points)** All pages implemented and routable (/ login, signup, home, lobby, game, leaderboard, history, update-profile)
- **(4 points)** Authentication: login/signup works correctly; auth state persists across page reloads (using cookies/tokens)
- **(4 points)** Forms: basic validation, error messages, password confirmation
- **(4 points)** Navigation: React Router used; no page reloads except on logout; navbar persists after login
- **(4 points)** Socket.IO integration: client connects; listens and emits events correctly

13.3 Backend Implementation (20 points)

- **(4 points)** Express server runs; routes are organized by feature (auth, rooms, stats)
- **(4 points)** MongoDB integration: User and Game schemas properly defined; data persists
- **(4 points)** Authentication endpoints: login and signup work; password handling is present
- **(4 points)** Socket.IO events: room management (join, leave), game state broadcasting
- **(4 points)** Error handling: graceful error messages; no unhandled promise rejections in logs

13.4 Game Logic & Real-Time Features (25 points)

- (5 points) Game board renders correctly; tokens display and update in real-time
- (5 points) Dice rolling on server; validated moves prevent cheating
- (5 points) Token movement logic: correct advancement, captures, home stretch rules
- (5 points) Win detection: first player to finish is recorded; rankings determined correctly
- (5 points) Real-time sync: all clients show identical board state; no latency issues; chat messages appear instantly

13.5 Database & Persistence (10 points)

- (3 points) User data: accounts created and updated; coins reflect game results
- (4 points) Game history: matches recorded with players, finishing order, and coin awards
- (3 points) Leaderboard: correctly sorted by coins; tiebreaker logic (fewer games) applied

13.6 Requirements Checklist Compliance (5 points)

- (5 points) All items in the **Requirements Checklist** section are completed as specified

13.7 Deductions

- -10 points: Code does not build or run after `npm install`
- -5 points: `node_modules/` folder included in submission
- -5 points: Missing environment variable setup instructions or `config.env.example`
- **Deduction as per item:** In Case of Incomplete requirements
- **Disciplinary Action:** Evidence of AI tool usage (e.g., identical code from ChatGPT, GitHub Copilot patterns)